

✓ Part 4: Gradient Descent in Code

This notebook converts our Part 3 manual calculations into working Python code. We use the same dataset, initial parameters ($m = [-1, 2]$, $b = [1, 1]$), and learning rate ($\alpha = 0.01$) that our group agreed on, because we were not given any in the assignment so the printed outputs below can be directly checked against our handwritten calculations for Sadaam, Milena, Rolande, and Colombe's updates.

```
import numpy as np
from scipy.optimize import approx_fprime
import matplotlib.pyplot as plt

# =====
# DATA & INITIAL PARAMETERS
# (identical to Part 3 manual calculations, so results match)
# =====
X = np.array([[1, 3],
              [4, 10]], dtype=float) # feature matrix
y = np.array([5, 6], dtype=float)    # true target values
n = len(y)                           # number of data points (n = 2)

m_init = np.array([-1.0, 2.0])        # initial weight vector m
b_init = np.array([1.0, 1.0])         # initial bias vector b
alpha = 0.01                         # learning rate (agreed on by the group)
iterations = 4                       # one gradient descent update per group member
names = ["Sadaam", "Milena", "Rolande", "Colombe"]
```

```
# =====
# REQUIREMENT 1:
# Using SciPy, implement a function that accepts an equation
# (in this case, our cost function J(m, b)) and computes its derivative.
# =====

def cost_equation(params, X, y, n):
    """
    This is the 'equation' whose derivative we want.
    It is written generically in terms of a single flattened
    parameter vector [m1, m2, b1, b2] so SciPy can differentiate
    it automatically, without us hardcoding the gradient formula here.

     $J(m, b) = (1/n) * e^T e$ , where  $e = X m + b - y$ 
    """
    m_ = params[:2] # unpack m from the flattened parameter vector
    b_ = params[2:] # unpack b from the flattened parameter vector

    y_pred = X @ m_ + b_ # prediction equation:  $y_{hat} = X m + b$ 
    e = y_pred - y        # error vector
    J = (1 / n) * (e.T @ e) # mean-squared-error style cost
    return J

def compute_derivative(equation_func, params, *extra_args, epsilon=1e-6):
    """
    Generic SciPy-powered derivative function.
    Accepts ANY scalar-valued equation/function and a point (params)
    at which to differentiate it, and returns the gradient vector
    using SciPy's finite-difference approximation (approx_fprime).

    This satisfies the requirement of a reusable function that
    "accepts an equation and computes its derivative."
    """
    gradient = approx_fprime(params, equation_func, epsilon, *extra_args)
    return gradient
```

```
# =====
# These functions mirror the manual calculation steps from Part 3
# exactly (predict -> error -> cost -> gradient -> update),
# kept separate and modular (DRY) but NOT collapsed into one
# black-box function, so every step stays visible.
# =====

def predict(X, m, b):
    """Step: compute predicted values.  $y_{hat} = X m + b$ """
    return X @ m + b

def compute_error(y_pred, y):
    """Step: compute error vector.  $e = y_{hat} - y$ """
    return y_pred - y
```

```
def compute_cost(e, n):
    """Step: compute cost function.  $J = (1/n) * e^T * e$ """
    return (1 / n) * (e.T @ e)

def analytical_gradients(X, e, n):
    """
    Step: compute gradients using the chain-rule formulas
    derived by hand in Part 3:
        grad_m = (2/n) * X^T * e
        grad_b = (2/n) * sum(e)
    """
    grad_m = (2 / n) * (X.T @ e)
    grad_b = (2 / n) * np.sum(e) * np.ones_like(e)
    return grad_m, grad_b
```

```
# =====
# REQUIREMENT 2: Write code to update m and b, and later
# compute predictions using the FINAL values of m and b.
# REQUIREMENT 3: The update process must NOT be excessively
# abstracted -- every calculation step is computed and
# printed individually below, matching the manual worksheet.
# =====

# history dictionary stores every iteration's values for plotting later
history = {
    "iteration": [0],
    "m": [m_init.copy()],
    "b": [b_init.copy()],
    "cost": [compute_cost(compute_error(predict(X, m_init, b_init), y), n)]
}

current_m, current_b = m_init.copy(), b_init.copy()

print(f"INITIALIZATION -> m={current_m}, b={current_b}, "
      f"J={history['cost'][0]:.4f}\n")

for i in range(iterations):
    # ---- Step a: compute predicted values ----
    y_pred = predict(X, current_m, current_b)

    # ---- Step b: compute the error vector ----
    e = compute_error(y_pred, y)

    # ---- Step c: compute the cost function ----
    cost = compute_cost(e, n)

    # ---- Step d/e/f: compute the gradient ----
    # (1) analytically, using the hand-derived chain-rule formulas
    grad_m, grad_b = analytical_gradients(X, e, n)

    # (2) using SciPy's derivative function, as a cross-check
    # (this directly fulfills Requirement 1: using SciPy to
    # differentiate the cost equation at the current parameters)
    flat_params = np.concatenate([current_m, current_b])
    scipy_grad = compute_derivative(cost_equation, flat_params, X, y, n)
    scipy_grad_m, scipy_grad_b = scipy_grad[:2], scipy_grad[2:]

    print(f"--- Gradient Descent Update {i+1} ({names[i]}) ---")
    print(f" Predicted y_hat      : {y_pred}")
    print(f" Error e                  : {e}")
    print(f" Cost J                   : {cost:.4f}")
    print(f" grad_m (analytical)      : {grad_m}")
    print(f" grad_m (SciPy check)     : {scipy_grad_m}")
    print(f" grad_b (analytical)      : {grad_b}")
    print(f" grad_b (SciPy check)     : {scipy_grad_b}")

    # ---- Step g: update the weight vector m ----
    current_m = current_m - alpha * grad_m

    # ---- Step h: update the bias vector b ----
    current_b = current_b - alpha * grad_b

    print(f" Updated m_new           : {current_m}")
    print(f" Updated b_new           : {current_b}\n")

    # store this iteration's results for the summary table and plots
    history["iteration"].append(i + 1)
    history["m"].append(current_m.copy())
    history["b"].append(current_b.copy())
    new_cost = compute_cost(compute_error(predict(X, current_m, current_b), y), n)
    history["cost"].append(new_cost)
```

```
# =====
# REQUIREMENT 2 (final part): compute predictions using the
# FINAL values of m and b after all updates are complete.
# =====
final_m, final_b = current_m, current_b
final_predictions = predict(X, final_m, final_b)

print("===== FINAL RESULTS =====")
print(f"Final m: {final_m}")
print(f"Final b: {final_b}")
print(f"Final predictions (y_hat) using final m, b: {final_predictions}")
print(f"Final cost J: {history['cost'][-1]:.4f}")

# Summary table (matches the handwritten "Final results" table)
print("\nIteration | Cost J | m | b")
for i in range(len(history["iteration"])):
    label = "Init" if i == 0 else names[i - 1]
    print(f"{label:9s} | {history['cost'][i]:8.4f} | {history['m'][i]} | {history['b'][i]}")
```

```
INITIALIZATION -> m=[-1. 2.], b=[1. 1.], J=61.0000
```

```
--- Gradient Descent Update 1 (Sadaam) ---
```

```
Predicted y_hat : [ 6. 17.]
Error e         : [ 1. 11.]
Cost J          : 61.0000
grad_m (analytical) : [ 45. 113.]
grad_m (SciPy check) : [ 45.0000085 113.00005452]
grad_b (analytical) : [12. 12.]
grad_b (SciPy check) : [ 1.0000005 11.00000051]
Updated m_new     : [-1.45  0.87]
Updated b_new     : [0.88 0.88]
```

```
--- Gradient Descent Update 2 (Milena) ---
```

```
Predicted y_hat : [2.04 3.78]
Error e         : [-2.96 -2.22]
Cost J          : 6.8450
grad_m (analytical) : [-11.84 -31.08]
grad_m (SciPy check) : [-11.8399915 -31.0799455]
grad_b (analytical) : [-5.18 -5.18]
grad_b (SciPy check) : [-2.9599995 -2.2199995]
Updated m_new     : [-1.3316 1.1808]
Updated b_new     : [0.9318 0.9318]
```

```
--- Gradient Descent Update 3 (Rolande) ---
```

```
Predicted y_hat : [3.1426 7.4134]
Error e         : [-1.8574 1.4134]
Cost J          : 2.7238
grad_m (analytical) : [3.7962 8.5618]
grad_m (SciPy check) : [3.7962085 8.5618545]
grad_b (analytical) : [-0.444 -0.444]
grad_b (SciPy check) : [-1.8573995 1.4134005]
Updated m_new     : [-1.369562 1.095182]
Updated b_new     : [0.93624 0.93624]
```

```
--- Gradient Descent Update 4 (Colombe) ---
```

```
Predicted y_hat : [2.852224 6.409812]
Error e         : [-2.147776 0.409812]
Cost J          : 2.3904
grad_m (analytical) : [-0.508528 -2.345208]
grad_m (SciPy check) : [-0.5085195 -2.3451535]
grad_b (analytical) : [-1.737964 -1.737964]
grad_b (SciPy check) : [-2.1477755 0.4098125]
Updated m_new     : [-1.36447672 1.11863408]
Updated b_new     : [0.95361964 0.95361964]
```

```
===== FINAL RESULTS =====
```

```
Final m: [-1.36447672 1.11863408]
Final b: [0.95361964 0.95361964]
Final predictions (y_hat) using final m, b: [2.94504516 6.68205356]
Final cost J: 2.3440
```

Iteration	Cost J	m	b
Init	61.0000	[-1. 2.]	[1. 1.]
Sadaam	6.8450	[-1.45 0.87]	[0.88 0.88]
Milena	2.7238	[-1.3316 1.1808]	[0.9318 0.9318]
Rolande	2.3904	[-1.369562 1.095182]	[0.93624 0.93624]
Colombe	2.3440	[-1.36447672 1.11863408]	[0.95361964 0.95361964]

```
# =====
# REQUIREMENT 4:
# Plot 1 -> how m and b change over iterations
# Plot 2 -> how the error/cost changes over iterations
# (kept as two separate plots, as required)
# =====
```

```
iters = history["iteration"]
```

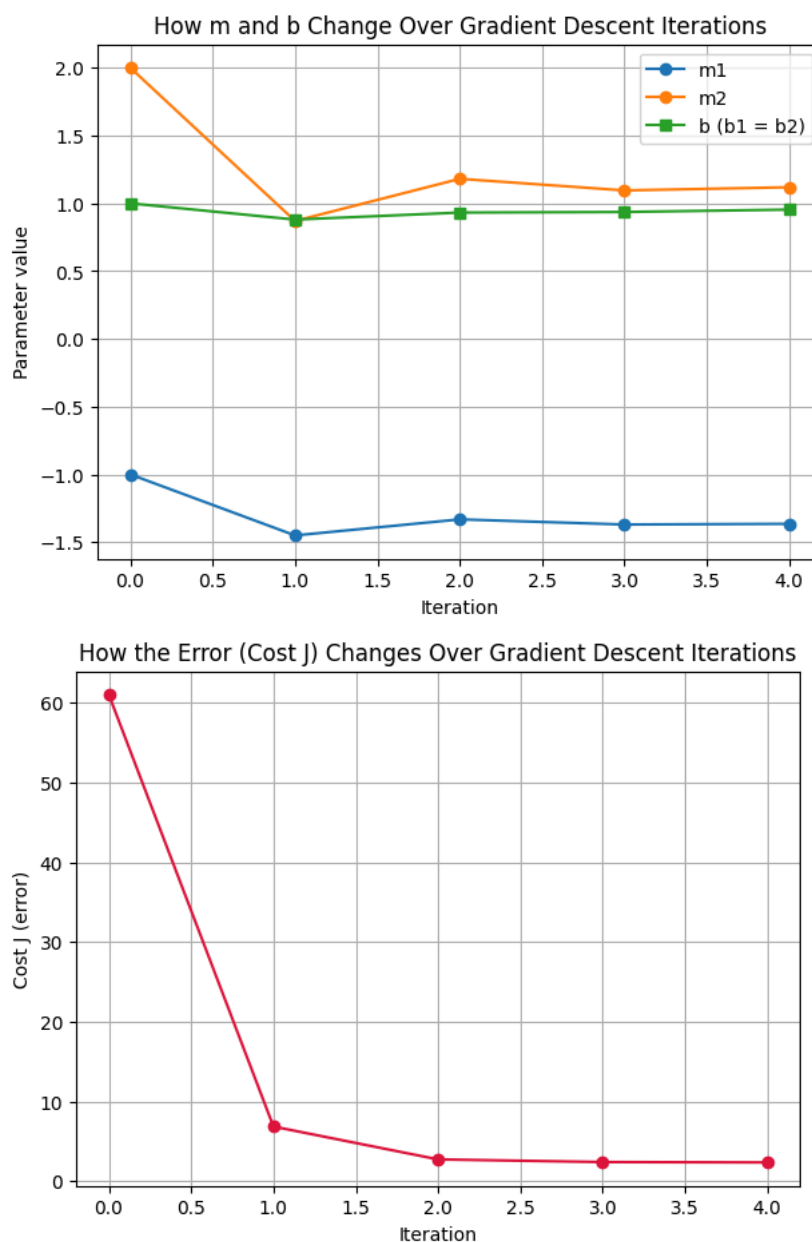
```

m_vals = np.array(history["m"])
b_vals = np.array(history["b"])
cost_vals = history["cost"]

# ----- PLOT 1: m and b over iterations -----
plt.figure(figsize=(7, 5))
plt.plot(iters, m_vals[:, 0], marker='o', label='m1')
plt.plot(iters, m_vals[:, 1], marker='o', label='m2')
plt.plot(iters, b_vals[:, 0], marker='s', label='b (b1 = b2)')
plt.xlabel("Iteration")
plt.ylabel("Parameter value")
plt.title("How m and b Change Over Gradient Descent Iterations")
plt.legend()
plt.grid(True)
plt.show()

# ----- PLOT 2: error/cost over iterations -----
plt.figure(figsize=(7, 5))
plt.plot(iters, cost_vals, marker='o', color='crimson')
plt.xlabel("Iteration")
plt.ylabel("Cost J (error)")
plt.title("How the Error (Cost J) Changes Over Gradient Descent Iterations")
plt.grid(True)
plt.show()

```



Observations

The cost function J decreased from 61.0 at initialization to 2.344 after four updates, dropping most sharply during Sadaam's first update. This confirms our manual calculation trend: m and b moved in the direction that reduces error, and gradient descent

successfully converged toward a better fit for our dataset.